

## ASSIGNMENT-17.5

HT.NO-2203A52100

### BATCH-50

#### TASK-1:

Loan Approval Dataset Cleaning

Use AI to generate preprocessing code for a loan dataset.

Instructions:

- Handle missing values in loan\_amount, credit\_score.
- Encode loan\_status (Approved/Rejected).
- Scale numerical features using standard scaling.
- Split the dataset into training and testing sets.

Sample Code:

```
import pandas as pd
data = pd.read_csv("loadapproval.csv")
print(data.head())
```

Dataset link:

<https://www.kaggle.com/datasets/amineipad/loan-approval-dataset>

Expected Output:

- A machine-learning-ready loan dataset.

#### CODE:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder, MinMaxScaler, OneHotEncoder
```

```
# --- TASK 1 – LOAN APPROVAL DATASET CLEANING --- print("---
TASK 1: LOAN APPROVAL DATASET CLEANING ---") # Load dataset
```

```
# Replace 'loan_approval_dataset.csv' with your actual file path try:
```

```

df_loan = pd.read_csv('loan_approval_dataset.csv')
print("Loan Approval Dataset loaded successfully.")
print("Initial shape:", df_loan.shape)  print("Initial
missing values:\n", df_loan.isnull().sum()) except
FileNotFoundError:

print("Error: 'loan_approval_dataset.csv' not found. Creating a dummy DataFrame for
demonstration.")  data = {

    'loan_amount': [10000, 20000, np.nan, 15000, 30000, 25000, np.nan, 12000],

    'credit_score': [700, 650, 720, np.nan, 750, 680, 690, np.nan],

    'loan_status': ['Approved', 'Rejected', 'Approved', 'Approved', 'Rejected', 'Approved',
'Rejected', 'Approved'],

    'age': [30, 45, 28, 35, 50, 40, 33, 29],

    'income': [50000, 60000, 55000, 70000, 80000, 65000, 52000, 58000]

}

df_loan = pd.DataFrame(data)

# Handle missing values # Fill

'loan_amount' with its mean if

'loan_amount' in df_loan.columns:

    mean_loan_amount = df_loan['loan_amount'].mean()

df_loan['loan_amount'].fillna(mean_loan_amount, inplace=True)  print(f'Filled
missing 'loan_amount' with mean: {mean_loan_amount:.2f}')

# Fill 'credit_score' with its median if

'credit_score' in df_loan.columns:

median_credit_score =

df_loan['credit_score'].median()

df_loan['credit_score'].fillna(median
_credit_score, inplace=True)

print(f'Filled missing 'credit_score'

```

with median:

```
{median_credit_score:.2f}")
```

```
print("Missing values after handling:\n", df_loan.isnull().sum())
```

```
# Encode target column 'loan_status' #
```

```
Convert 'Approved' to 1 and 'Rejected' to 0
```

```
if 'loan_status' in df_loan.columns:
```

```
    df_loan['loan_status'] = df_loan['loan_status'].map({'Approved': 1, 'Rejected': 0})
```

```
print("Encoded 'loan_status' column (Approved=1, Rejected=0).")    print("Sample  
of encoded 'loan_status':\n", df_loan['loan_status'].value_counts())
```

```
# Identify numerical features for scaling (excluding the target column)
```

```
# Select only numerical columns for scaling, excluding 'loan_status' if it's already encoded as  
numerical numerical_features_loan =
```

```
df_loan.select_dtypes(include=np.number).columns.tolist()
```

```
if 'loan_status' in numerical_features_loan: # If loan_status was int/float and part of  
numerical columns    numerical_features_loan.remove('loan_status')
```

```
if numerical_features_loan:
```

```
    # Scale numerical features using StandardScaler
```

```
scaler_loan = StandardScaler()
```

```
    df_loan[numerical_features_loan] =
```

```
scaler_loan.fit_transform(df_loan[numerical_features_loan])    print("Scaled  
numerical features using StandardScaler.")    print("Sample of scaled  
data:\n", df_loan[numerical_features_loan].head()) else:
```

```
    print("No numerical features found for scaling in Loan Approval Dataset.")
```

```
# Split dataset into training and testing sets (80/20) if
```

```
'loan_status' in df_loan.columns and len(df_loan) > 1:
```

```

X = df_loan.drop('loan_status', axis=1)  y =
df_loan['loan_status']

X_train_loan, X_test_loan, y_train_loan, y_test_loan = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y if y.nunique() > 1 else None)  print(f"Dataset split into
training (80%) and testing (20%) sets.")  print(f"X_train shape: {X_train_loan.shape},
X_test shape: {X_test_loan.shape}")  print(f"y_train shape: {y_train_loan.shape}, y_test
shape: {y_test_loan.shape}") else:

    print("Cannot split data: 'loan_status' column missing or dataset too small.")

print("\n" + "="*60 + "\n")

```

## OUTPUT:

```

... --- TASK 1: LOAN APPROVAL DATASET CLEANING ---
Error: 'loan_approval_dataset.csv' not found. Creating a dummy DataFrame for demonstration.
Filled missing 'loan_amount' with mean: 18666.67
Filled missing 'credit_score' with median: 695.00
Missing values after handling:
  loan_amount    0
  credit_score    0
  loan_status    0
  age            0
  income         0
dtype: int64
Encoded 'loan_status' column (Approved=1, Rejected=0).
Sample of encoded 'loan_status':
  loan_status
1         5
0         3
Name: count, dtype: int64
Scaled numerical features using StandardScaler.
Sample of scaled data:
   loan_amount  credit_score    age    income
0   -1.407463    0.092057 -0.831948 -1.197978
1    0.216533   -1.749092  1.164727 -0.133109
2    0.000000    0.828517 -1.098171 -0.665544
3   -0.595465   -0.092057 -0.166390  0.931761
4    1.840529    1.933207  1.830285  1.996631
Dataset split into training (80%) and testing (20%) sets.
X_train shape: (6, 4), X_test shape: (2, 4)
y_train shape: (6,), y_test shape: (2,)

```

## TASK-2:

### Task 2 – Social Media Profiles

Preprocess a social media engagement dataset using AI assistance.

Instructions:

- Remove duplicate user profiles.
- Handle missing values • Encode categorical columns

Dataset link: <https://www.kaggle.com/datasets/medaxone/top-100-social-media-profiles>

Expected Output:

- A cleaned dataset for engagement prediction.

**CODE:**

```
# --- TASK 2 – SOCIAL MEDIA PROFILES DATASET --- print("---
TASK 2: SOCIAL MEDIA PROFILES DATASET ---")

# Load dataset

# Replace 'social_media_profiles.csv' with your actual file path try:

df_social = pd.read_csv('social_media_profiles.csv')

print("Social Media Profiles Dataset loaded successfully.")

print("Initial shape:", df_social.shape)    print("Initial
missing values:\n", df_social.isnull().sum()) except
FileNotFoundError:

    print("Error: 'social_media_profiles.csv' not found. Creating a dummy DataFrame for
demonstration.")    data = {

        'user_id': [1, 2, 3, 1, 4, 5, 6, 7],

        'username': ['userA', 'userB', 'userC', 'userA', 'userD', 'userE', 'userF', 'userG'],

        'age': [25, 30, np.nan, 25, 35, 28, 40, np.nan],

        'gender': ['Male', 'Female', 'Other', 'Male', 'Female', np.nan, 'Male', 'Female'],

        'country': ['USA', 'Canada', 'UK', 'USA', 'Germany', 'France', 'USA', 'Canada'],

        'followers': [100, 200, 150, 100, 300, 120, 250, 180]

    }

    df_social = pd.DataFrame(data)

# Remove duplicate user profiles based on a unique identifier like 'user_id' or 'username'

initial_rows_social = df_social.shape[0] df_social.drop_duplicates(subset=['user_id'],
inplace=True) print(f"Removed {initial_rows_social - df_social.shape[0]} duplicate
user profiles.") print("Shape after removing duplicates:", df_social.shape)
```

```

# Handle missing values (using appropriate method: mean/median for numerical, mode for
categorical) for col in df_social.columns: if df_social[col].isnull().any():

    if df_social[col].dtype == 'object' or pd.api.types.is_categorical_dtype(df_social[col]): #
Categorical

        mode_val = df_social[col].mode()[0] # mode() can return multiple values, take the
first
        df_social[col].fillna(mode_val, inplace=True) print(f'Filled missing
'{col}' with mode: {mode_val}')
    elif pd.api.types.is_numeric_dtype(df_social[col]):
# Numerical
        if df_social[col].nunique() > 10: # Heuristic for continuous-like
numbers, use mean
            mean_val = df_social[col].mean()
df_social[col].fillna(mean_val, inplace=True) print(f'Filled missing '{col}' with
mean: {mean_val:.2f}')
        else: # Discrete-like numbers, use median
            median_val = df_social[col].median()
df_social[col].fillna(median_val,
inplace=True) print(f'Filled missing '{col}' with median: {median_val:.2f}')
print("Missing values after handling:\n", df_social.isnull().sum())

# Encode categorical columns (using One-Hot Encoding for 'gender' and 'country', Label
Encoding for 'username' if unique counts are high)

# Identify categorical columns categorical_cols_social =
df_social.select_dtypes(include='object').columns.tolist()

for col in categorical_cols_social:

    if df_social[col].nunique() > 5 and col != 'username': # Heuristic for high cardinality, might
consider target encoding for more advanced cases

        # For demonstration, using OneHotEncoder for columns with more than 5 unique values
but not too many unique values to avoid sparse matrices

        # For 'username', we might not want to encode it directly as it's an identifier

    if col not in ['user_id', 'username']:

        print(f'Applying One-Hot Encoding to '{col}'.')
        encoder =
OneHotEncoder(handle_unknown='ignore', sparse_output=False)
        encoded_features = encoder.fit_transform(df_social[[col]])

```

```

        encoded_df = pd.DataFrame(encoded_features,
columns=encoder.get_feature_names_out([col]), index=df_social.index)
df_social = pd.concat([df_social.drop(columns=[col]), encoded_df], axis=1)    elif
col not in ['user_id', 'username']:

    print(f'Applying Label Encoding to '{col}'.')

le = LabelEncoder()    df_social[col] =

le.fit_transform(df_social[col])    else:

    print(f'Skipping encoding for '{col}' (identifier column). Example:
{df_social[col].head(2).tolist()}")

print("Sample of processed Social Media data (first 5 rows):\n", df_social.head())

print("\n" + "="*60 + "\n")

```

## OUTPUT:

```

...
=====

--- TASK 2: SOCIAL MEDIA PROFILES DATASET ---
Error: 'social_media_profiles.csv' not found. Creating a dummy DataFrame for demonstration.
Removed 1 duplicate user profiles.
Shape after removing duplicates: (7, 6)
Filled missing 'age' with median: 30.00
Filled missing 'gender' with mode: Female
Missing values after handling:
  user_id    0
  username    0
  age         0
  gender      0
  country     0
  followers   0
dtype: int64
Skipping encoding for 'username' (identifier column). Example: ['userA', 'userB']
Applying Label Encoding to 'gender'.
Applying Label Encoding to 'country'.
Sample of processed Social Media data (first 5 rows):
   user_id username  age  gender  country  followers
0         1   userA  25.0       1        4         100
1         2   userB  30.0       0        0         200
2         3   userC  30.0       2        3         150
4         4   userD  35.0       0        2         300
5         5   userE  28.0       0        1         120

=====

```

## TASK-3:

Task 3 – Weather Dataset Feature Engineering

Task:

Use AI to preprocess a weather dataset for forecasting.

Instructions:

- Convert date column into datetime format.
- Handle missing values in temperature, humidity.
- Create new features (season, week\_number).
- Normalize rainfall values.

Dataset link: <https://www.kaggle.com/code/alsamaualalhinai/weather-dataset/input>

Expected Output:

- A processed dataset ready for forecasting models.

### **CODE:**

```
# --- TASK 3 – WEATHER DATASET FEATURE ENGINEERING ---
```

```
print("--- TASK 3: WEATHER DATASET FEATURE ENGINEERING ---")
```

```
# Load dataset
```

```
# Replace 'weather_dataset.csv' with your actual file path try:
```

```
df_weather = pd.read_csv('weather_dataset.csv')
```

```
print("Weather Dataset loaded successfully.") print("Initial
```

```
shape:", df_weather.shape) print("Initial missing
```

```
values:\n", df_weather.isnull().sum()) except
```

```
FileNotFoundError:
```

```
print("Error: 'weather_dataset.csv' not found. Creating a dummy DataFrame for demonstration.") data = {
```

```
    'date': ['2023-01-01', '2023-01-02', '2023-02-15', '2023-03-10', '2023-04-20', '2023-0505',  
            '2023-06-01', '2023-07-10', '2023-08-25', '2023-09-15', '2023-10-01', '2023-11-11', '202312-  
            20', '2024-01-05'],
```

```
    'temperature': [10.5, 11.2, np.nan, 15.1, 18.0, 22.5, 25.0, np.nan, 28.1, 20.3, 14.5, 8.9,  
                    3.1, 5.0],
```

```
    'humidity': [80, np.nan, 75, 70, 65, np.nan, 60, 85, 78, 82, 90, 92, 88, 85],
```

```
    'rainfall': [5.0, 2.1, 0.0, 1.5, 0.0, 10.2, 3.5, 0.0, 0.0, 6.7, 12.0, 8.1, 4.0, 1.0]
```



```

    }

    df_weather = pd.DataFrame(data)

# Convert 'date' column to datetime format if
'date' in df_weather.columns:
    df_weather['date'] = pd.to_datetime(df_weather['date'])
print("Converted 'date' column to datetime format.")
print("Sample dates:\n", df_weather['date'].head()) else:
    print("Warning: 'date' column not found in Weather Dataset.")

# Handle missing values # Fill
'temperature' with its mean if
'temperature' in df_weather.columns:
    mean_temperature = df_weather['temperature'].mean()
df_weather['temperature'].fillna(mean_temperature, inplace=True)    print(f'Filled
missing 'temperature' with mean: {mean_temperature:.2f}')

# Fill 'humidity' with its median if
'humidity' in df_weather.columns:
    median_humidity = df_weather['humidity'].median()
df_weather['humidity'].fillna(median_humidity, inplace=True)    print(f'Filled
missing 'humidity' with median: {median_humidity:.2f}')

print("Missing values after handling:\n", df_weather.isnull().sum())

# Create new features
# Create 'season' based on month
# Assuming Northern Hemisphere seasons: Dec-Feb (Winter), Mar-May (Spring), Jun-Aug
(Summer), Sep-Nov (Autumn) if
'date' in df_weather.columns:

```

```

df_weather['month'] = df_weather['date'].dt.month
def get_season(month):    if month in [12, 1, 2]:
    return 'Winter'
elif month in [3, 4, 5]:
    return 'Spring'
elif month in [6, 7, 8]:
    return 'Summer'
else:
    return 'Autumn'
df_weather['season'] =
df_weather['month'].apply(get_season)    print("Created 'season' feature based on
month.")    print("Sample 'season' values:\n", df_weather[['date', 'month',
'season']].head())    df_weather.drop('month', axis=1, inplace=True) # Drop
auxiliary month column

# Create 'week_number' from date
df_weather['week_number'] = df_weather['date'].dt.isocalendar().week.astype(int) #
.isocalendar() returns a DataFrame, .week extracts the series    print("Created
'week_number' feature from date.")    print("Sample 'week_number' values:\n",
df_weather[['date', 'week_number']].head()) else:
    print("Cannot create 'season' and 'week_number' features as 'date' column is missing or not
in datetime format.")

# Normalize 'rainfall' using MinMaxScaler if 'rainfall' in
df_weather.columns and df_weather['rainfall'].nunique() > 1:
    scaler_rainfall = MinMaxScaler()    df_weather['rainfall_normalized'] =
scaler_rainfall.fit_transform(df_weather[['rainfall']])    print("Normalized 'rainfall' using
MinMaxScaler.")
    print("Sample of normalized 'rainfall':\n", df_weather[['rainfall',
'rainfall_normalized']].head()) else:
    print("Cannot normalize 'rainfall': column missing or all values are the same.")

```

```
print("Sample of processed Weather data (first 5 rows):\n", df_weather.head())
```

```
print("\n" + "="*60 + "\n")
```

## OUTPUT:

```
... --- TASK 3: WEATHER DATASET FEATURE ENGINEERING ---
Error: 'weather_dataset.csv' not found. Creating a dummy DataFrame for demonstration.
Converted 'date' column to datetime format.
Sample dates:
  0    2023-01-01
  1    2023-01-02
  2    2023-02-15
  3    2023-03-10
  4    2023-04-20
Name: date, dtype: datetime64[ns]
Filled missing 'temperature' with mean: 15.18
Filled missing 'humidity' with median: 81.00
Missing values after handling:
  date      0
  temperature  0
  humidity    0
  rainfall    0
dtype: int64
Created 'season' feature based on month.
Sample 'season' values:
   date month season
0 2023-01-01     1  Winter
1 2023-01-02     1  Winter
2 2023-02-15     2  Winter
3 2023-03-10     3   Spring
4 2023-04-20     4   Spring
Created 'week_number' feature from date.
Sample 'week_number' values:
   date week_number
```

## TASK-4:

### Task 4 – Movie Ratings Dataset Preparation

Task:

Clean and preprocess a movie ratings dataset using AI- generated scripts.

Instructions:

- Handle missing values in rating, genre.
- Encode categorical variables like genre, language.
- Remove duplicate movie entries.
- Scale rating values between 0 and 1.

Dataset link:

<https://www.kaggle.com/datasets/PromptCloudHQ/imdb- data>

Expected Output:

- A dataset suitable for recommendation systems.

### **CODE:**

```
# --- TASK 4 – MOVIE RATINGS DATASET PREPROCESSING --- print("---
TASK 4: MOVIE RATINGS DATASET PREPROCESSING ---")

# Load dataset

# Replace 'imdb_movie_dataset.csv' with your actual file path

try:

    df_movie = pd.read_csv('imdb_movie_dataset.csv')

print("IMDB Movie Dataset loaded successfully.")

print("Initial shape:", df_movie.shape)    print("Initial
missing values:\n", df_movie.isnull().sum()) except
FileNotFoundError:

    print("Error: 'imdb_movie_dataset.csv' not found. Creating a dummy DataFrame for
demonstration.")    data = {

        'movie_id': [1, 2, 3, 4, 1, 5, 6, 7],

        'title': ['Movie A', 'Movie B', 'Movie C', 'Movie D', 'Movie A', 'Movie E', 'Movie F',
'Movie G'],

        'rating': [8.5, 7.2, np.nan, 9.0, 8.5, 6.8, np.nan, 7.5],

        'genre': ['Action', 'Comedy', 'Drama', 'Action', 'Action', 'Sci-Fi', 'Comedy', np.nan],

        'language': ['English', 'Spanish', 'English', 'French', 'English', 'English', np.nan, 'German'],

        'year': [2020, 2021, 2022, 2020, 2020, 2023, 2021, 2022]

    }

    df_movie = pd.DataFrame(data)
```

```

# Handle missing values # Fill
'rating' with its mean if 'rating'
in df_movie.columns:
    mean_rating = df_movie['rating'].mean()
df_movie['rating'].fillna(mean_rating, inplace=True)    print(f'Filled missing
'rating' with mean: {mean_rating:.2f}')

# Fill 'genre' with its mode if
'genre' in df_movie.columns:
    mode_genre = df_movie['genre'].mode()[0]
df_movie['genre'].fillna(mode_genre, inplace=True)    print(f'Filled
missing 'genre' with mode: {mode_genre}')

print("Missing values after handling:\n", df_movie.isnull().sum())

# Encode categorical columns 'genre' and 'language' categorical_cols_movie
= ['genre', 'language']

for col in categorical_cols_movie:    if col in df_movie.columns
and df_movie[col].dtype == 'object':
    if df_movie[col].nunique() < 10: # Use OneHotEncoder for columns with fewer unique
categories        print(f'Applying One-Hot Encoding to '{col}'.')        encoder =
OneHotEncoder(handle_unknown='ignore', sparse_output=False)        encoded_features
= encoder.fit_transform(df_movie[[col]])

    encoded_df = pd.DataFrame(encoded_features,
columns=encoder.get_feature_names_out([col]), index=df_movie.index)
df_movie = pd.concat([df_movie.drop(columns=[col]), encoded_df], axis=1)

    else: # For higher cardinality, LabelEncoder is an option, but OneHot is often preferred if
cardinality is not excessive

```

```

        print(f'Applying Label Encoding to '{col}' (high cardinality or many unique values).
Generally One-Hot is preferred if not too many categories.")        le = LabelEncoder()
df_movie[col] = le.fit_transform(df_movie[col])    elif col not in df_movie.columns:

        print(f'Warning: Column '{col}' not found in Movie Ratings Dataset.")

# Remove duplicate movie entries based on 'movie_id' or 'title' initial_rows_movie =
df_movie.shape[0] df_movie.drop_duplicates(subset=['movie_id'], inplace=True)
print(f'Removed {initial_rows_movie - df_movie.shape[0]} duplicate movie entries.")
print("Shape after removing duplicates:", df_movie.shape)

# Scale rating values between 0 and 1 using MinMaxScaler if
'rating' in df_movie.columns and df_movie['rating'].nunique() > 1:
    scaler_movie_rating = MinMaxScaler()    df_movie['rating_scaled'] =
scaler_movie_rating.fit_transform(df_movie[['rating']])    print("Scaled 'rating'
values between 0 and 1 using MinMaxScaler.")    print("Sample of scaled 'rating':\n",
df_movie[['rating', 'rating_scaled']].head()) else:
    print("Cannot scale 'rating': column missing or all values are the same.")

print("Sample of processed Movie Ratings data (first 5 rows):\n", df_movie.head())

print("\n" + "="*60 + "\n")

print("All tasks completed. Please ensure you replace dummy file paths with your actual
dataset paths.")

```

**OUTPUT:**

```

...
--- TASK 4: MOVIE RATINGS DATASET PREPROCESSING ---
Error: 'imdb_movie_dataset.csv' not found. Creating a dummy DataFrame for demonstration.
Filled missing 'rating' with mean: 7.92
Filled missing 'genre' with mode: Action
Missing values after handling:
  movie_id    0
  title      0
  rating     0
  genre      0
  language   1
  year       0
  dtype: int64
Applying One-Hot Encoding to 'genre'.
Applying One-Hot Encoding to 'language'.
Removed 1 duplicate movie entries.
Shape after removing duplicates: (7, 13)
Scaled 'rating' values between 0 and 1 using MinMaxScaler.
Sample of scaled 'rating':
   rating  rating_scaled
0  8.500000      0.772727
1  7.200000      0.181818
2  7.916667      0.507576
3  9.000000      1.000000
5  6.800000      0.000000
Sample of processed Movie Ratings data (first 5 rows):
   movie_id  title  rating  year  genre_Action  genre_Comedy  genre_Drama  \
0         1  Movie A  8.500000  2020           1.0           0.0           0.0
1         2  Movie B  7.200000  2021           0.0           1.0           0.0

```